fundmat
O blog da materia "Fundamentos de Matemáticas"

Programación en Sage

Publicado o 04/09/2014 por franpena

Un *script* es un conjunto de comandos de Sage guardados en un fichero `.sage` para ser ejecutados posteriormente. Por ejemplo, imaginemos que queremos representar funciones con `plot` al tiempo que se salvan en un fichero; podemos guardar la función `plotNsave` en el fichero `my_plots.sage`:

```
def plotNsave(f, a, b, file):
    """
    Aqui se pone la descripcion de plotNsave
    Esta informacion se muestra al escribir:  plotNsave?
    """
    p = plot(f, (a, b))
    show(p)
    p.save(file)
```

Este script puede ejecutarse en un libro Jupyter o una terminal de Sage:

```
sage: pwd             # muestra la carpeta actual
u'/home/user'
sage: cd 'Escritorio' # cambio a una carpeta hija
/home/user/Escritorio
sage: ls              # muestra el contenido
['my_plots.sage']
sage: load('my_plots.sage')
sage: cd ..           # vuelve a la carpeta padre
```

Cuando se trabaja en una hoja `.sagews` de Cocalc, los comandos anteriores deben sustituirse por `os.getcwd()`, `os.chdir('Escritorio')` y `os.listdir('.')`, respectivamente. Usando `attach` en vez de `load`, la **terminal** de Sage reconocerá automáticamente los cambios que hagamos en el fichero con un editor externo. Una vez cargado, su contenido puede ser usado normalmente:

```
sage: f(x) = x^2
sage: plotNsave(f, 1, 2, 'p.png')
```

Funciones lambda

Las funciones lambda permiten crear funciones anónimas compuestas de una única expresión. Son, básicamente, una forma escueta de definir una función:

```
sage: h = lambda x,y: (x^2,y^2)
sage: h(1,2)
```

```
(1, 4)
```

Son cómodos en ciertos casos, por ejemplo, para dar los valores de una matriz:

```
sage: A = matrix(RDF, 10, 10, lambda i,j: 1/(i+j+1))
```

Bucles

El bucle `for` ejecuta una serie de comandos para todos los valores de una lista:

```
fact = 1
for i in (2..10):
    fact = fact*i
```

Al final del bucle, `fact` contendrá $10! = \prod_{n=1}^{10} n$. Los comandos internos al bucle se sangrarán con espacios en blanco. Los bucles pueden anidarse con sangrado creciente.

El bucle `while` ejecuta una serie de comandos mientras se dé una condición:

```
suma = RDF(0)
n = RDF(1)
term = 1/n^4
while term > 1e-14:
   suma += term
   n += 1
   term = 1/n^4
```

Al final del bucle, `sum` contendrá una aproximación de $\sum_{n=1}^{\infty} \frac{1}{n^4}$, calculada sumando aquellos términos mayores que la *tolerancia* dada, 1e-14.

Sólo usaremos el bucle `while` en vez de un `for` cuando el número de iteraciones a realizar sea desconocido a priori.

Al recorrer listas, puede interesar recorrer sus índices, sus valores o una combinación. Por ejemplo, si sólo se debe trabajar con un vector, pueden recorrerse sus valores:

```
v = [10,20,30]
suma = 0
for vi in v:
    suma += vi
```

Si se debe trabajar con varios vectores, puede usarse el índice:

```
v = [10,20,30]
w = [40, 50, 60]
z = []
for i in range(len(v)):
   z.append(v[i]/w[i])
```

A menudo existe una opción más concisa, usando *comprehension list* y `zip` para devolver una agrupación de valores:

```
z = [vi/wi for vi,wi in zip(v,w)]
```

De este modo, tenemos varias opciones para construir nuestras funciones. Pongamos por caso, el construir una función que devuelva una lista con las componentes de x al cuadrado.

```
def f(x):
  l = []
  for i in range(len(x)):
    l.append(x[i]^2)
  return l
```

Podemos obtener un código más conciso con *comprehension lists*:

```
def f(x):
  l = [x[i]^2 for i in range(len(x))]
return l
```

Finalmente, es posible definir la función en una sola línea con lambda:

```
f = lambda x: [xi^2 for xi in x]
```

Decisiones

Las estructuras `if` permiten decidir en función de una o varias condiciones:

```
i = 0 # asignar a i un valor
if i>0:
    print 'positivo'
elif i<0:
    print 'negativo'
else:
    print 'nulo'
```

El bloque `elif` y `else` son opcionales. Puede haber varios bloques `elif`.

También es posible usar un if ternario:

```
v = True if 2>3 else False
```

Este if se puede incluir en una *comprehension list*:

```
z = [vi/wi for vi,wi in zip(v,w) if wi != 0]
```

Otros comandos

El comando `continue` interrumpe la ejecución de la iteración actual de un bucle `for` o `while`. El comando `break` interrumpe la ejecución del bucle `for` o `while` más interno. Véase por ejemplo, la búsqueda del índice de la posición del primer elemento negativo de la lista `x`:

```
x = [1,0,-1, 2]
for i in range(len(x)):
    if x[i]<0:
        break
print 'El indice es '+str(i)
```

El comando `return` termina la ejecución de una función e indica el valor de su respuesta, como puede verse en la defición de la función `f` en esta entrada.

Esta entrada foi publicada en Sage. Ligazón permanente.

1 Response to *Programación en Sage*

Pingback: *Expresiones simbólicas en Sage | fundmat*

fundmat

Crea un sitio web ou un blogue de balde en WordPress.com.